

C++ Standard Library Issues to be moved in Virtual Plenary, Jul. 2022

Doc. no. P2618R0

Date: 2022-07-15

Audience: WG21

Reply to: Jonathan Wakely <lwgchair@gmail.com>

- [Ready Issues](#)

Ready Issues

The committee is requested to approve the proposed resolutions for the issues in this document.

These issues are in **Ready** status, meaning the LWG has reached consensus that the issue is a defect in the Standard, the **Proposed Resolution** is correct, and the issue is ready to forward to the full committee for approval.

[3564](#). `transform_view::iterator<true>::value_type` and `iterator_category` should use `const F&`

Section: 26.7.7.3 [[range.transform.iterator](#)] **Status:** [Ready](#) **Submitter:** Tim Song **Opened:** 2021-06-06 **Last modified:** 2022-07-15

Priority: 2

View all other [issues](#) in [`range.transform.iterator`].

Discussion:

Iterators obtained from a `const transform_view` invoke the transformation function as `const`, but the `value_type` and `iterator_category` determination uses plain `F&`, i.e., non-`const`.

[2021-06-14; Reflector poll]

Set priority to 2 after reflector poll, send to SG9 for design clarification. Should `r` and `as_const(r)` guarantee same elements?

[2022-07-08; Reflector poll]

SG9 has decided to proceed with this PR in its 2021-09-13 telecon and not block the issue for a paper on the more general `const/non-const` problem.

Set status to Tentatively Ready after five votes in favour during reflector poll.

[2022-07-15; LWG telecon: move to Ready]

Proposed resolution:

This wording is relative to [N4885](#).

1. Modify 26.7.7.3 [[range.transform.iterator](#)] as indicated:

```
namespace std::ranges {
    template<input_range V, copy_constructible F>
        requires view<V> && is_object_v<F> &&
            regular_invocable<F&, range_reference_t<V>> &&
                can_reference<invoke_result_t<F&, range_reference_t<V>>>
    template<bool Const>
    class transform_view<V, F>::iterator {
    private:
        [...]
    public:
        using iterator_concept = see below;
        using iterator_category = see below; // not always present
        using value_type =
            remove_cvref_t<invoke_result_t<maybe-const<Const, F>&,
                range_reference_t<Base>>>;
        using difference_type = range_difference_t<Base>;
        [...]
    };
};
```

-1- [...]

-2- The member `typedef-name iterator_category` is defined if and only if `Base` models `forward_range`. In that case, `iterator::iterator_category` is defined as follows: Let `c` denote the type `iterator_traits<iterator_t<Base>>::iterator_category`.

- (2.1) — If `is_lvalue_reference_v<invoke_result_t<maybe_const<Const, F>&, range_reference_t<Base>>>` is true, then
- (2.1.1) — [...]
- (2.1.2) — [...]
- (2.2) — Otherwise, `iterator_category` denotes `input_iterator_tag`.
-

3617. function/package_task deduction guides and deducing this

Section: 22.10.17.3.2 [\[func.wrap.func.con\]](#), 33.10.10.2 [\[futures.task.members\]](#) **Status:** [Ready](#) **Submitter:** Barry Revzin **Opened:** 2021-10-09 **Last modified:** 2022-07-15

Priority: 2

View other [active issues](#) in [\[func.wrap.func.con\]](#).

View all other [issues](#) in [\[func.wrap.func.con\]](#).

Discussion:

With the adoption of deducing this ([P0847](#)), we can now create types whose call operator is an explicit object member function, which means that `decltype(&F::operator())` could have pointer-to-function type rather than pointer-to-member-function type. This means that the deduction guides for `std::function` (22.10.17.3.2 [\[func.wrap.func.con\]](#)) and `std::package_task` (33.10.10.2 [\[futures.task.members\]](#)) will simply fail:

```
struct F {
    int operator()(this const F&) { return 42; }
};

std::function g = F{}; // error: decltype(&F::operator()) is not of the form R(G::*)(A...) cv &opt noexcept_opt
```

We should update the deduction guides to keep them in line with the core language.

[2021-10-14; Reflector poll]

Set priority to 2 after reflector poll.

Previous resolution [SUPERSEDED]:

This wording is relative to [N4892](#).

1. Modify 22.10.17.3.2 [\[func.wrap.func.con\]](#) as indicated:

```
template<class F> function(F) -> function<see below>;

-16- Constraints: &F::operator() is well-formed when treated as an unevaluated operand and
decltype(&F::operator()) is either of the form R(G::*)(A...) cv &opt noexcept_opt or of the form
R(*)(G cv &opt, A...) noexcept_opt for a class type G.

-17- Remarks: The deduced type is function<R(A...)>.
```

2. Modify 33.10.10.2 [\[futures.task.members\]](#) as indicated:

```
template<class F> package_task(F) -> package_task<see below>;

-7- Constraints: &F::operator() is well-formed when treated as an unevaluated operand and
decltype(&F::operator()) is either of the form R(G::*)(A...) cv &opt noexcept_opt or of the form
R(*)(G cv &opt, A...) noexcept_opt for a class type G.

-8- Remarks: The deduced type is package_task<R(A...)>.
```

[2021-10-17; Improved wording based on Tim Song's suggestion]

[2022-07-01; Reflector poll]

Set status to Tentatively Ready after nine votes in favour during reflector poll.

[2022-07-15; LWG telecon: move to Ready]

Proposed resolution:

This wording is relative to [N4892](#).

1. Modify 22.10.17.3.2 [\[func.wrap.func.con\]](#) as indicated:

```
template<class F> function(F) -> function<see below>;
```

-16- *Constraints:* `&F::operator()` is well-formed when treated as an unevaluated operand and `decltype(&F::operator())` is **either** of the form `R(G::*)(A...)` `cv &_opt noexcept_opt` **or of the form** `R(*)(G, A...)` `noexcept_opt` for a **class** type `G`.

-17- *Remarks:* The deduced type is `function<R(A...)>`.

2. Modify 33.10.10.2 [\[futures.task.members\]](#) as indicated:

```
template<class F> packaged_task(F) -> packaged_task<see below>;
```

-7- *Constraints:* `&F::operator()` is well-formed when treated as an unevaluated operand and `decltype(&F::operator())` is **either** of the form `R(G::*)(A...)` `cv &_opt noexcept_opt` **or of the form** `R(*)(G, A...)` `noexcept_opt` for a **class** type `G`.

-8- *Remarks:* The deduced type is `packaged_task<R(A...)>`.

3656. Inconsistent bit operations returning a count

Section: 22.15.5 [\[bit.pow.two\]](#) **Status:** [Ready](#) **Submitter:** Nicolai Josuttis **Opened:** 2021-12-30 **Last modified:** 2022-07-15

Priority: 3

Discussion:

Among the bit operations returning a count, `bit_width()` is the only one not returning an `int`.

This has the following consequences:

```
std::uint64_t b64 = 1;
b64 = std::rotr(b64, 1);
int count1 = std::popcount(b64);    // OK
int count2 = std::countl_zero(b64);  // OK
int count3 = std::bit_width(b64);    // OOPS
```

The last line may result in a warning such as:

Warning: conversion from long long unsigned to int may change value

You have to use a `static_cast` to avoid the warning.

Note that [P0553R4](#) explicitly states the following design considerations, which I would also assume to apply to the later added functions from [P0556R3](#):

The counting operations return "int" quantities, consistent with the rule "use an int unless you need something else". This choice does not reflect, in the type, the fact that counts are always non-negative.

[2022-01-30; Reflector poll]

Set priority to 3 after reflector poll. Eight votes for P0, but request LEWG confirmation before setting it to Tentatively Ready.

[2022-02-22 LEWG telecon; Status changed: LEWG → Open]

No objection to unanimous consent to send the proposed resolution for LWG3656 to LWG for C++23. The changes in P1956 changed the functions to be more counting than mathematical.

[2022-07-08; Reflector poll]

Set status to Tentatively Ready after ten votes in favour during reflector poll.

[2022-07-15; LWG telecon: move to Ready]

Proposed resolution:

This wording is relative to [N4901](#).

1. Modify 22.15.2 [\[bit.syn\]](#), header `<bit>` synopsis, as indicated:

```
[...]
template<class T>
constexpr int bit_width(T x) noexcept;
[...]
```

2. Modify 22.15.5 [\[bit.pow.two\]](#), as indicated:

```
template<class T>
constexpr int bit_width(T x) noexcept;
```

-11- *Constraints:* `T` is an unsigned integer type (6.8.2 [\[basic.fundamental\]](#)).

3659. Consider ATOMIC_FLAG_INIT undeprecation

Section: D.26.5 [\[depr.atomics.flag\]](#) Status: [Ready](#) Submitter: Aaron Ballman Opened: 2022-01-18 Last modified: 2022-07-15

Priority: 3

Discussion:

[P0883R2](#) deprecated ATOMIC_VAR_INIT and ATOMIC_FLAG_INIT largely based on rationale from WG14. However, WG14 only deprecated ATOMIC_VAR_INIT because we were told by implementers that ATOMIC_FLAG_INIT is still necessary for some platforms (platforms for which "clear" is actually not all zero bits, which I guess exist).

I'd like to explore whether WG21 should undeprecate ATOMIC_FLAG_INIT as there was no motivation that I could find in the paper on the topic or in the discussion at [P0883R0 \[Jacksonville 2018\]](#).

One possible approach would be to undeprecate it from `<stdatomic.h>` only (C++ can still use the constructors from `<atomic>` and shared code can use the macros from `<stdatomic.h>`).

[2022-01-30; Reflector poll]

Set priority to 3 after reflector poll. Send to SG1.

[2022-07-06; SG1 confirm the direction, Jonathan adds wording]

"In response to LWG 3659, add ATOMIC_FLAG_INIT to `<stdatomic.h>` as undeprecated."

SFFN ASA
3 00 0 0

"In response to LWG 3659, undeprecate ATOMIC_FLAG_INIT in `<atomic>`."

SFFN ASA
2 31 0 0

[2022-07-11; Reflector poll]

Set status to Tentatively Ready after six votes in favour during reflector poll.

[2022-07-15; LWG telecon: move to Ready]

Proposed resolution:

This wording is relative to [N4910](#).

1. Modify 33.5.2 [\[atomics.syn\]](#), Header `<atomic>` synopsis, as indicated:

```
void atomic_flag_notify_all(volatile atomic_flag*) noexcept;
void atomic_flag_notify_all(atomic_flag*) noexcept;
#define ATOMIC_FLAG_INIT see below

// [atomics.fences], fences
extern "C" void atomic_thread_fence(memory_order) noexcept;
extern "C" void atomic_signal_fence(memory_order) noexcept;
```

2. Move the content of D.26.5 [\[depr.atomics.flag\]](#) from Annex D to the end of 33.5.10 [\[atomics.flag\]](#).

```
#define ATOMIC_FLAG_INIT see below
```

Remarks: The macro ATOMIC_FLAG_INIT is defined in such a way that it can be used to initialize an object of type atomic_flag to the clear state. The macro can be used in the form:

```
atomic_flag guard = ATOMIC_FLAG_INIT;
```

It is unspecified whether the macro can be used in other initialization contexts. For a complete static-duration object, that initialization shall be static.

3. Modify 33.5.12 [\[stdatomic.h.syn\]](#) C compatibility, as indicated:

```
using std::atomic_flag_clear;           // see below
using std::atomic_flag_clear_explicit;  // see below
#define ATOMIC_FLAG_INIT see below

using std::atomic_thread_fence;         // see below
using std::atomic_signal_fence;        // see below
```

4. Modify D.26.1 [\[depr.atomics.general\]](#) in Annex D as indicated:

```
#define ATOMIC_VAR_INIT(value) see below
```

```
#define ATOMIC_FLAG_INIT see below  
}
```

3670. Cpp17InputIterators don't have integer-class difference types

Section: 26.6.4.3 [[range.iota.iterator](#)] **Status:** [Ready](#) **Submitter:** Casey Carter **Opened:** 2022-02-04 **Last modified:** 2022-07-15

Priority: Not Prioritized

View all other [issues](#) in [[range.iota.iterator](#)].

Discussion:

26.6.4.3 [[range.iota.iterator](#)] defines

```
using iterator_category = input_iterator_tag; // present only if W models incrementable
```

but when difference_type is an integer-class type the iterator does not meet the *Cpp17InputIterator* requirements.

[2022-02-07; Daniel comments]

As requested by LWG [3376](#) and wording implemented by [P2393R1](#), integer-class types are no longer required to have class type.

Previous resolution [SUPERSEDED]:

This wording is relative to [N4901](#).

1. Modify 26.6.4.3 [[range.iota.iterator](#)], class `iota_view::iterator` synopsis, as indicated:

```
namespace std::ranges {  
    template<weakly_incrementable W, semiregular Bound>  
        requires weakly-equality-comparable-with<W, Bound> && copyable<W>  
        struct iota_view<W, Bound>::iterator {  
            [...]   
            using iterator_category = input_iterator_tag; // present only if W models incrementable and  
                                                         // IOTA-DIFF-T(W) is not a class type  
  
            using value_type = W;  
            using difference_type = IOTA-DIFF-T(W);  
            [...]   
        };  
}
```

[2022-02-07; Casey Carter provides improved wording]

[2022-03-04; Reflector poll]

Set status to Tentatively Ready after seven votes in favour during reflector poll.

[2022-07-15; LWG telecon: move to Ready]

Proposed resolution:

This wording is relative to [N4901](#).

1. Modify 26.6.4.3 [[range.iota.iterator](#)], class `iota_view::iterator` synopsis, as indicated:

```
namespace std::ranges {  
    template<weakly_incrementable W, semiregular Bound>  
        requires weakly-equality-comparable-with<W, Bound> && copyable<W>  
        struct iota_view<W, Bound>::iterator {  
            [...]   
            using iterator_category = input_iterator_tag; // present only if W models incrementable and  
                                                         // IOTA-DIFF-T(W) is an integral type  
  
            using value_type = W;  
            using difference_type = IOTA-DIFF-T(W);  
            [...]   
        };  
}
```

3671. atomic_fetch_xor missing from stdatomic.h

Section: 33.5.12 [[stdatomic.h.syn](#)] **Status:** [Ready](#) **Submitter:** Hubert Tong **Opened:** 2022-02-09 **Last modified:** 2022-07-15

Priority: Not Prioritized

Discussion:

[C17 subclause 7.17.7.5](#) provides `atomic_fetch_xor` and `atomic_fetch_xor_explicit`. `stdatomic.h` in the working draft ([N4901](#)) does not.

[2022-02-09; Jonathan comments and provides wording]

C++20 33.5.2 [[atomics.syn](#)] has both of them, too, so it should definitely be in the common subset.

[2022-03-04; Reflector poll]

Set status to Tentatively Ready after eight votes in favour during reflector poll.

[2022-07-15; LWG telecon: move to Ready]

Proposed resolution:

This wording is relative to [N4901](#).

1. Modify 33.5.12 [[stdatomic.h.syn](#)], header `<stdatomic.h>` synopsis, as indicated:

[Drafting Note: The editor is kindly requested to reorder these "atomic_fetch_KEY" declarations to match the other synopses in Clause 33.5 [[atomics](#)]: add, sub, and, or, xor.]

```
[...]
using std::atomic_fetch_or;                // see below
using std::atomic_fetch_or_explicit;        // see below
using std::atomic_fetch_xor;                // see below
using std::atomic_fetch_xor_explicit;        // see below
using std::atomic_fetch_and;                // see below
using std::atomic_fetch_and_explicit;        // see below
[...]
```

3672. `common_iterator::operator->()` should return by value

Section: 25.5.4.4 [[common.iter.access](#)] **Status:** [Ready](#) **Submitter:** Jonathan Wakely **Opened:** 2022-02-12 **Last modified:** 2022-07-15

Priority: Not Prioritized

View all other issues in [[common.iter.access](#)].

Discussion:

25.5.4.4 [[common.iter.access](#)] p5 (5.1) says that `common_iterator<T*, S>::operator->()` returns `std::get<T*>(v_)` which has type `T* const&`. That means that `iterator_traits::pointer` is `T* const&` as well (this was recently clarified by LWG [3660](#)). We have an actual pointer here, why are we returning it by reference?

For the three bullet points in 25.5.4.4 [[common.iter.access](#)] p5, the second and third return by value anyway, so `decltype(auto)` is equivalent to `auto`. For the first bullet, it would make a lot more sense for raw pointers to be returned by value. That leaves the case where the iterator has an `operator->()` member, which could potentially benefit from returning by reference. But it must return something that is iterator-like or pointer-like, which we usually just pass by value. Casey suggested we should just change `common_iterator<I, S>::operator->()` to return by value in all cases.

Libstdc++ has always returned by value, as an unintended consequence of using a union instead of `std::variant<I, S>`, so that it doesn't use `std::get<I>` to return the member.

[2022-03-04; Reflector poll]

Set status to Tentatively Ready after six votes in favour during reflector poll.

[2022-07-15; LWG telecon: move to Ready]

Proposed resolution:

This wording is relative to [N4901](#).

1. Modify 25.5.4.1 [[common.iterator](#)], class template `common_iterator` synopsis, as indicated:

```
[...]
constexpr decltype(auto) operator*();
constexpr decltype(auto) operator*() const
    requires dereferenceable<const I>;
constexpr decltype(auto) operator->() const
    requires see below;
[...]
```

2. Modify 25.5.4.4 [[common.iter.access](#)] as indicated:

```
constexpr decltype(auto) operator->() const
requires see below;
```

-3- The expression in the *requires*-clause is equivalent to: [...]

-4- *Preconditions*: `holds_alternative<I>(v_)` is true.

-5- *Effects*:

(5.1) — If `I` is a pointer type or if the expression `get<I>(v_).operator->()` is well-formed, equivalent to: `return get<I>(v_);`

(5.2) — Otherwise, if `iter_reference_t<I>` is a reference type, equivalent to:

```
auto&& tmp = *get<I>(v_);
return addressof(tmp);
```

(5.3) — Otherwise, equivalent to: `return proxy(*get<I>(v_));` where *proxy* is the exposition-only class:

```
class proxy {
    iter_value_t<I> keep_;
    constexpr proxy(iter_reference_t<I>&& x)
        : keep_(std::move(x)) {}
public:
    constexpr const iter_value_t<I>* operator->() const noexcept {
        return addressof(keep_);
    }
};
```

3683. operator== for polymorphic_allocator cannot deduce template argument in common cases

Section: 20.4.3 [[mem.poly.allocator.class](#)] Status: [Ready](#) Submitter: Pablo Halpern Opened: 2022-03-18 Last modified: 2022-07-15

Priority: Not Prioritized

View all other [issues](#) in [[mem.poly.allocator.class](#)].

Discussion:

In `<memory_resource>`, the equality comparison operator for `pmr::polymorphic_allocator` is declared in namespace scope as:

```
template<class T1, class T2>
bool operator==(const polymorphic_allocator<T1>& a,
               const polymorphic_allocator<T2>& b) noexcept;
```

Since `polymorphic_allocator` is implicitly convertible from `memory_resource*`, one would naively expect — and the author of `polymorphic_allocator` intended — the following code to work:

```
std::pmr::unsynchronized_pool_resource pool_rsrc;
std::pmr::vector<int> vec(&pool_rsrc); // Converts to std::pmr::polymorphic_allocator<int>
[...]
assert(vec.get_allocator() == &pool_rsrc); // (1) Compare polymorphic_allocator to memory_resource*
```

Unfortunately, the line labeled (1) is ill-formed because the type `T2` in `operator==` cannot be deduced.

Possible resolution 1 (PR1) is to supply a second `operator==`, overloaded for comparison to `memory_resource*`:

```
template<class T1, class T2>
bool operator==(const polymorphic_allocator<T1>& a,
               const polymorphic_allocator<T2>& b) noexcept;
template<class T>
bool operator==(const polymorphic_allocator<T>& a,
               memory_resource* b) noexcept;
```

The rules for implicitly defined spaceship and comparison operators obviates defining `operator!=` or `operator==(b, a)`. This PR would allow `polymorphic_allocator` to be compared for equality with `memory_resource*`, but not with any other type that is convertible to `polymorphic_allocator`.

Possible resolution 2 (PR2) is to replace `operator==` with a homogeneous version where type deduction occurs only for one template parameter:

```
template<class T1, class T2>
bool operator==(const polymorphic_allocator<T1>& a,
               const polymorphic_allocator<T2>& b) noexcept;
template<class T>
bool operator==(const polymorphic_allocator<T>& a,
               const type_identity_t<polymorphic_allocator<T>>& b) noexcept;
```

This version will work with any type that is convertible to `polymorphic_allocator`.

Possible resolution 3 (PR3), the proposed resolution, below, is to add a homogeneous equality operator as a "hidden friend", such that it is found by ADL only if one argument is a `polymorphic_allocator` and the other argument is convertible to `polymorphic_allocator`. As with PR2, this PR will work

with any type that is convertible to `polymorphic_allocator`.

Note to reader: Proof of concept for the three possible resolutions can be seen at [this godbolt link](#). Uncomment one of PR1, PR2, or PR3 macros to see the effects of each PR.

[2022-05-17; Reflector poll]

Set status to Tentatively Ready after six votes in favour during reflector poll.

[2022-07-15; LWG telecon: move to Ready]

Proposed resolution:

This wording is relative to [N4901](#).

1. Modify 20.4.3 [\[mem.poly_allocator.class\]](#), class template `polymorphic_allocator` synopsis, as indicated:

```
namespace std::pmr {
    template<class Tp = byte> class polymorphic_allocator {
        memory_resource* memory_rsrc; // exposition only

    public:
        using value_type = Tp;

        [...]
        memory_resource* resource() const;

        // friends
        friend bool operator==(const polymorphic_allocator& a,
                               const polymorphic_allocator& b) noexcept {
            return *a.resource() == *b.resource();
        }
    };
}
```

[3687](#). `expected<cv void, E>` move constructor should move

Section: 22.8.7.4 [\[expected.void.assign\]](#) Status: [Ready](#) Submitter: Jonathan Wakely Opened: 2022-03-23 Last modified: 2022-07-15

Priority: Not Prioritized

Discussion:

For `expected<cv void>::operator=(expected&&)` we have this in the last bullet of the *Effects* element:

Otherwise, equivalent to `unex = rhs.error()`.

That should be a move assignment, not a copy assignment.

[2022-05-17; Reflector poll]

Set status to Tentatively Ready after eight votes in favour during reflector poll.

[2022-07-15; LWG telecon: move to Ready]

Proposed resolution:

This wording is relative to [N4910](#).

1. Modify 22.8.7.4 [\[expected.void.assign\]](#) as indicated:

```
constexpr expected& operator=(expected&& rhs) noexcept(see below);
```

-4- *Effects*:

(4.1) — If `this->has_value()` && `rhs.has_value()` is true, no effects.

(4.2) — Otherwise, if `this->has_value()` is true, equivalent to:

```
construct_at(addressof(unex), std::move(rhs.unex));
has_val = false;
```

(4.3) — Otherwise, if `rhs.has_value()` is true, destroys `unex` and sets `has_val` to true.

(4.4) — Otherwise, equivalent to `unex = std::move(rhs.error())`.

3692. `zip_view::iterator`'s `operator<=>` is overconstrained

Section: 26.7.20.3 [\[range.zip.iterator\]](#) Status: [Ready](#) Submitter: S. B. Tam Opened: 2022-04-21 Last modified: 2022-07-15

Priority: Not Prioritized

Discussion:

`zip_view::iterator`'s `operator<=>` is constrained with `(three_way_comparable<iterator_t<maybe-const<Const, Views>>> && ...)`. This is unnecessary, because the comparison is performed on the stored *tuple-or-pair*, and both `std::tuple` and `std::pair` provide `operator<=>` regardless of whether the elements are three-way comparable.

Note that, because neither `std::tuple` nor `std::pair` provides `operator<` since C++20, comparing two `zip::iterators` with `operator<` (which is specified to return `x.current_ < y.current_`, where *current_* is a *tuple-or-pair*) eventually uses *tuple* or *pair*'s `operator<=>` anyway.

Thus, I think it's possible to make `operator<=>` not require `three_way_comparable`. This also makes it possible to remove the operator functions for `<`, `>`, `<=`, `>=` and rely on the operators synthesized from `operator<=>`.

[2022-04-24; Daniel comments and provides wording]

It should be pointed out that by still constraining `operator<=>` with `all-random-access<Const, Views...>` we also constrain by `random_access_iterator<iterator_t<maybe-const<Const, Views>>>` which again means that we constrain by `totally_ordered<iterator_t<maybe-const<Const, Views>>>`, so this operator will only be satisfied with iterators *I* that satisfy *partially-ordered-with<I, I>*. Based on this argument the delegation to *tuple-or-pair*'s `operator<=>` that solely depends on *synth-three-way* should be appropriate.

[2022-05-17; Reflector poll]

Set status to Tentatively Ready after six votes in favour during reflector poll.

[2022-07-15; LWG telecon: move to Ready]

Proposed resolution:

This wording is relative to [N4910](#).

1. Modify 26.7.20.3 [\[range.zip.iterator\]](#) as indicated:

```
namespace std::ranges {
    [...]
    template<input_range... Views>
        requires (view<Views> && ...) && (sizeof...(Views) > 0)
    template<bool Const>
    class zip_view<Views...>::iterator {
        tuple-or-pair<iterator_t<maybe-const<Const, Views>>>...> current_; // exposition only
        constexpr explicit iterator(tuple-or-pair<iterator_t<maybe-const<Const, Views>>>...>); // exposition only
    public:
        [...]
        friend constexpr bool operator==(const iterator& x, const iterator& y)
            requires (equality_comparable<iterator_t<maybe-const<Const, Views>>> && ...);

        friend constexpr bool operator<(const iterator& x, const iterator& y)
            requires all_random_access<Const, Views...>;
        friend constexpr bool operator>(const iterator& x, const iterator& y)
            requires all_random_access<Const, Views...>;
        friend constexpr bool operator<=(const iterator& x, const iterator& y)
            requires all_random_access<Const, Views...>;
        friend constexpr bool operator>=(const iterator& x, const iterator& y)
            requires all_random_access<Const, Views...>;
        friend constexpr auto operator<=>(const iterator& x, const iterator& y)
            requires all-random-access<Const, Views...> &&
            three_way_comparable<iterator_t<maybe-const<Const, Views>>> && ...);
        [...]
    };
    [...]
}

[...]
```

~~friend constexpr bool operator<(const iterator& x, const iterator& y)~~
~~requires all_random_access<Const, Views...>;~~

~~-16- Returns: x.current_ < y.current_-~~

~~friend constexpr bool operator>(const iterator& x, const iterator& y)~~
~~requires all_random_access<Const, Views...>;~~

~~-17- Effects: Equivalent to: return y < x;~~

~~friend constexpr bool operator<=(const iterator& x, const iterator& y)~~
~~requires all_random_access<Const, Views...>;~~

~~-18- Effects: Equivalent to: return !(y < x);~~

```
friend constexpr bool operator==(const iterator& x, const iterator& y)  
requires all-random-access<Const, Views...>;
```

~~-19- Effects: Equivalent to: return ! (x < y);~~

```
friend constexpr auto operator<=(const iterator& x, const iterator& y)  
requires all-random-access<Const, Views...> &&  
(three_way_comparable<iterator_t<maybe_const<Const, Views>>> && ...);
```

~~-20- Returns: x.current_ <= y.current_.~~

3701. Make formatter<remove_cvref_t<const charT[N]>, charT> requirement explicit

Section: 22.14.6.2 [format.formatter.spec] Status: [Ready](#) Submitter: Mark de Wever Opened: 2022-05-17 Last modified: 2022-07-15

Priority: Not Prioritized

View other [active issues](#) in [format.formatter.spec].

View all other [issues](#) in [format.formatter.spec].

Discussion:

The wording in 22.14.5 [format.functions]/20 and 22.14.5 [format.functions]/25 both contain

formatter<remove_cvref_t<T_i>, charT> meets the *BasicFormatter* requirements (22.14.6.1 [formatter.requirements]) for each T_i in Args.

The issue is that remove_cvref_t<const charT[N]> becomes charT[N]. 22.14.6.2 [format.formatter.spec]/2.2 requires a specialization for

```
template<size_t N> struct formatter<const charT[N], charT>;
```

but there's no requirement to provide

```
template<size_t N> struct formatter<charT[N], charT>;
```

There's no wording preventing library vendors from providing additional specializations. So it's possible to implement the current specification but the indirect requirement is odd. I noticed this while implementing a formattable concept. The concept is based on the formattable concept of [P2286](#) "Formatting Ranges" (This paper is targeting C++23.)

It could be argued that the specialization

```
template<size_t N> struct formatter<const charT[N], charT>
```

is not needed and should be removed from the Standard. This will be an API break. Vendors can decide to keep the no longer required specialization as an extension; which would lead to implementation divergence. Microsoft is already shipping this specialization as stable and Victor doesn't like the removal too.

Therefore I only propose to add the required formatter specialization.

[2022-06-21; Reflector poll]

Set status to Tentatively Ready after seven votes in favour during reflector poll.

[2022-07-15; LWG telecon: move to Ready]

Proposed resolution:

This wording is relative to [N4910](#).

1. Modify 22.14.6.2 [format.formatter.spec] as indicated:

-2- Let charT be either char or wchar_t. Each specialization of formatter is either enabled or disabled, as described below. Each header that declares the template formatter provides the following enabled specializations:

(2.1) — The specializations [...]

(2.2) — For each charT, the string type specializations

```
template<> struct formatter<charT*, charT>;  
template<> struct formatter<const charT*, charT>;  
template<size_t N> struct formatter<charT[N], charT>;  
template<size_t N> struct formatter<const charT[N], charT>;  
template<class traits, class Allocator>  
    struct formatter<basic_string<charT, traits, Allocator>, charT>;  
template<class traits>  
    struct formatter<basic_string_view<charT, traits>, charT>;
```

(2.3) — [...]

3702. Should `zip_transform_view::iterator` remove operator<?

Section: 26.7.21.3 [[range.zip.transform.iterator](#)] **Status:** [Ready](#) **Submitter:** Hewill Kang **Opened:** 2022-05-21 **Last modified:** 2022-07-15

Priority: Not Prioritized

Discussion:

After LWG [3692](#), `zip_view::iterator` only provides `operator<=>`. Since the comparison of `zip_transform_view::iterator` uses `zip_view::iterator`'s `operator<=>`, it is possible to remove `zip_transform_view::iterator`'s `operator<`, `>`, `<=`, `>=` and just detect if `ziperator`'s `operator<=>` is available.

Since the `ziperator`'s `operator<=>` is valid only when `zip_view` is a `random_access_range`, we don't need to additionally constrain the `ziperator` to be `three_way_comparable`.

[2022-06-21; Reflector poll]

Set status to Tentatively Ready after five votes in favour during reflector poll.

[2022-07-15; LWG telecon: move to Ready]

Proposed resolution:

This wording is relative to [N4910](#).

1. Modify 26.7.21.3 [[range.zip.transform.iterator](#)] as indicated:

```
namespace std::ranges {
[...]
template<copy_constructible F, input_range... Views>
requires (view<Views> && ...) && (sizeof...(Views) > 0) && is_object_v<F> &&
    regular_invocable<F&, range_reference_t<Views>...> &&
    can_reference_invoke_result_t<F&, range_reference_t<Views>...>
template<bool Const>
class zip_transform_view<F, Views...>::iterator {
    using Parent = maybe_const<Const, zip_transform_view>;           // exposition only
    using Base = maybe_const<Const, InnerView>;                     // exposition only
    Parent* parent_ = nullptr;                                       // exposition only
    ziperator<Const> inner_;                                         // exposition only

    constexpr iterator(Parent& parent, ziperator<Const> inner);      // exposition only
public:
    [...]
    friend constexpr bool operator==(const iterator& x, const iterator& y)
        requires equality_comparable<ziperator<Const>>;

    friend constexpr bool operator<(const iterator& x, const iterator& y)
        requires random_access_range<Base>;
    friend constexpr bool operator>(const iterator& x, const iterator& y)
        requires random_access_range<Base>;
    friend constexpr bool operator<=(const iterator& x, const iterator& y)
        requires random_access_range<Base>;
    friend constexpr bool operator>=(const iterator& x, const iterator& y)
        requires random_access_range<Base>;
    friend constexpr auto operator<=>(const iterator& x, const iterator& y)
        requires random_access_range<Base> && three_way_comparable<ziperator<Const>>;
    [...]
};
[...]
```

[...]

```
friend constexpr bool operator==(const iterator& x, const iterator& y)
    requires equality_comparable<ziperator<Const>>;
friend constexpr bool operator<(const iterator& x, const iterator& y)
    requires random_access_range<Base>;
friend constexpr bool operator>(const iterator& x, const iterator& y)
    requires random_access_range<Base>;
friend constexpr bool operator<=(const iterator& x, const iterator& y)
    requires random_access_range<Base>;
friend constexpr bool operator>=(const iterator& x, const iterator& y)
    requires random_access_range<Base>;
friend constexpr auto operator<=>(const iterator& x, const iterator& y)
    requires random_access_range<Base> && three_way_comparable<ziperator<Const>>;
```

-14- Let *op* be the operator.

-15- *Effects*: Equivalent to: `return x.inner_ op y.inner_;`

3703. Missing requirements for `expected<T, E>` requires `is_void<T>`

Section: 22.8.7.1 [\[expected.void.general\]](#) **Status:** [Ready](#) **Submitter:** Casey Carter **Opened:** 2022-05-24 **Last modified:** 2022-07-15

Priority: 2

Discussion:

The partial specialization `expected<T, E>` requires `is_void<T>` specified in 22.8.7.1 [\[expected.void.general\]](#) is missing some template parameter requirements that should have been copied from 22.8.6.1 [\[expected.object.general\]](#). We should copy the pertinent requirements from the first two paragraphs of the latter subclause into new paragraphs in the first subclause (the pertinent requirement from the third paragraph is already present in 22.8.7.1 [\[expected.void.general\]](#)).

[2022-06-21; Jonathan adds "[Member](#)" before "`has_val`"]

[2022-06-21; Reflector poll]

Set priority to 2 after reflector poll.

[2022-06-21; Reflector poll]

Set status to Tentatively Ready after five votes in favour during reflector poll.

[2022-07-15; LWG telecon: move to Ready]

Proposed resolution:

This wording is relative to [N4910](#).

[Drafting note: There is some drive-by cleanup that I couldn't resist while touching this wording: (1) strike the redundant "suitably aligned" guarantee, (2) Don't repeat in prose that the exposition-only members are exposition-only.]

1. Modify 22.8.6.1 [\[expected.object.general\]](#) as indicated:

-1- Any object of type `expected<T, E>` either contains a value of type `T` or a value of type `E` within its own storage. Implementations are not permitted to use additional storage, such as dynamic memory, to allocate the object of type `T` or the object of type `E`. ~~These objects are allocated in a region of the `expected<T, E>` storage suitably aligned for the types `T` and `E`. Members `has_val`, `val`, and `unex` are provided for exposition only.~~ [Member](#) `has_val` indicates whether the `expected<T, E>` object contains an object of type `T`.

2. Modify 22.8.7.1 [\[expected.void.general\]](#) as indicated:

~~?- Any object of type `expected<T, E>` either represents a value of type `T`, or contains a value of type `E` within its own storage. Implementations are not permitted to use additional storage, such as dynamic memory, to allocate the object of type `E`. Member `has_val` indicates whether the `expected<T, E>` object represents a value of type `T`.~~

~~?- A program that instantiates the definition of the template `expected<T, E>` with a type for the `E` parameter that is not a valid template argument for `unexpected` is ill-formed.~~

-1- `E` shall meet the requirements of `Cpp17Destructible` (Table [tab:cpp17.destructible]).

3704. LWG 2059 added overloads that might be ill-formed for sets

Section: 24.4.6.1 [\[set.overview\]](#), 24.4.7.1 [\[multiset.overview\]](#), 24.5.6.1 [\[unord.set.overview\]](#), 24.5.7.1 [\[unord.multiset.overview\]](#) **Status:** [Ready](#)
Submitter: Jonathan Wakely **Opened:** 2022-05-25 **Last modified:** 2022-07-15

Priority: Not Prioritized

Discussion:

The restored `erase(iterator)` overloads introduced by LWG [2059](#) would be duplicates of the `erase(const_iterator)` ones if `iterator` and `const_iterator` are the same type, which is allowed for sets.

We should constrain them (or add prose) so that the `erase(iterator)` overloads are only present when the iterator types are distinct.

This applies to `set`, `multiset`, `unordered_set`, `unordered_multiset` (and `flat_set` and `flat_multiset`).

[2022-06-21; Reflector poll]

Set status to Tentatively Ready after seven votes in favour during reflector poll.

[2022-07-15; LWG telecon: move to Ready]

Proposed resolution:

This wording is relative to [N4910](#).

1. Modify 24.4.6.1 [\[set.overview\]](#), class template set synopsis, as indicated:

```
iterator erase(iterator position) requires (!same as<iterator, const iterator>);
iterator erase(const_iterator position);
```

2. Modify 24.4.7.1 [\[multiset.overview\]](#), class template multiset synopsis, as indicated:

```
iterator erase(iterator position) requires (!same as<iterator, const iterator>);
iterator erase(const_iterator position);
```

3. Modify 24.5.6.1 [\[unord.set.overview\]](#), class template unordered_set synopsis, as indicated:

```
iterator erase(iterator position) requires (!same as<iterator, const iterator>);
iterator erase(const_iterator position);
```

4. Modify 24.5.7.1 [\[unord.multiset.overview\]](#), class template unordered_multiset synopsis, as indicated:

```
iterator erase(iterator position) requires (!same as<iterator, const iterator>);
iterator erase(const_iterator position);
```

3705. Hashability shouldn't depend on basic_string's allocator

Section: 23.4.6 [\[basic.string.hash\]](#) **Status:** [Ready](#) **Submitter:** Casey Carter **Opened:** 2022-05-26 **Last modified:** 2022-07-15

Priority: Not Prioritized

View all other [issues](#) in [\[basic.string.hash\]](#).

Discussion:

23.4.6 [\[basic.string.hash\]](#) says:

```
template<> struct hash<string>;
template<> struct hash<u8string>;
template<> struct hash<u16string>;
template<> struct hash<u32string>;
template<> struct hash<wstring>;
template<> struct hash<pmr::string>;
template<> struct hash<pmr::u8string>;
template<> struct hash<pmr::u16string>;
template<> struct hash<pmr::u32string>;
template<> struct hash<pmr::wstring>;
```

-1- If S is one of these string types, SV is the corresponding string view type, and s is an object of type S, then hash<S>()(s) == hash<SV>()(SV(s))

Despite that the hash value of a basic_string object is equivalent to the hash value of a corresponding basic_string_view object, which has no allocator, the capability to hash a basic_string depends on its allocator. All of the enabled specializations have specific allocators, which fact becomes more clear if we expand the type aliases:

```
template<> struct hash<basic_string<char, char_traits<char>, allocator<char>>>;
template<> struct hash<basic_string<char8_t, char_traits<char8_t>, allocator<char8_t>>>;

template<> struct hash<basic_string<char16_t, char_traits<char16_t>, allocator<char16_t>>>;
template<> struct hash<basic_string<char32_t, char_traits<char32_t>, allocator<char32_t>>>;

template<> struct hash<basic_string<wchar_t, char_traits<wchar_t>, allocator<wchar_t>>>;

template<> struct hash<basic_string<char, char_traits<char>, pmr::polymorphic_allocator<char>>>;
template<> struct hash<basic_string<char8_t, char_traits<char8_t>, pmr::polymorphic_allocator<char8_t>>>;

template<> struct hash<basic_string<char16_t, char_traits<char16_t>, pmr::polymorphic_allocator<char16_t>>>;
template<> struct hash<basic_string<char32_t, char_traits<char32_t>, pmr::polymorphic_allocator<char32_t>>>;

template<> struct hash<basic_string<wchar_t, char_traits<wchar_t>, pmr::polymorphic_allocator<wchar_t>>>;
```

If the hash value doesn't depend on the allocator type, why should we care about the allocator type? I posit that we should not, and that these ten explicit specializations should be replaced by 5 partial specializations that enable hashing basic_string specializations using these combinations of character type and traits type with any allocator type.

[2022-06-21; Reflector poll]

Set status to Tentatively Ready after seven votes in favour during reflector poll.

[2022-07-15; LWG telecon: move to Ready]

Proposed resolution:

This wording is relative to [N4910](#).

1. Modify 23.4.2 [\[string.syn\]](#), header `<string>` synopsis, as indicated:

```
[...]
// 23.4.6 \[basic.string.hash\], hash support
template<class T> struct hash;
template<> struct hash<string>;
template<> struct hash<u8string>;
template<> struct hash<u16string>;
template<> struct hash<u32string>;
template<> struct hash<wstring>;
template<> struct hash<pmr::string>;
template<> struct hash<pmr::u8string>;
template<> struct hash<pmr::u16string>;
template<> struct hash<pmr::u32string>;
template<> struct hash<pmr::wstring>;
template<class A> struct hash<basic string<char, char traits<char>, A>>;
template<class A> struct hash<basic string<char8\_t, char traits<char8\_t>, A>>;
template<class A> struct hash<basic string<char16\_t, char traits<char16\_t>, A>>;
template<class A> struct hash<basic string<char32\_t, char traits<char32\_t>, A>>;
template<class A> struct hash<basic string<wchar\_t, char traits<wchar\_t>, A>>;
[...]
```

2. Modify 23.4.6 [\[basic.string.hash\]](#) as indicated:

```
template<> struct hash<string>;
template<> struct hash<u8string>;
template<> struct hash<u16string>;
template<> struct hash<u32string>;
template<> struct hash<wstring>;
template<> struct hash<pmr::string>;
template<> struct hash<pmr::u8string>;
template<> struct hash<pmr::u16string>;
template<> struct hash<pmr::u32string>;
template<> struct hash<pmr::wstring>;
template<class A> struct hash<basic string<char, char traits<char>, A>>;
template<class A> struct hash<basic string<char8\_t, char traits<char8\_t>, A>>;
template<class A> struct hash<basic string<char16\_t, char traits<char16\_t>, A>>;
template<class A> struct hash<basic string<char32\_t, char traits<char32\_t>, A>>;
template<class A> struct hash<basic string<wchar\_t, char traits<wchar\_t>, A>>;
```

-1- If S is one of these string types, SV is the corresponding string view type, and s is an object of type S, then `hash<S>()`
`(s) == hash<SV>()(SV(s))`

[3707](#). `chunk_view::outer-iterator::value_type::size` should return unsigned type

Section: 26.7.24.4 [\[range.chunk.outer.value\]](#) **Status:** [Ready](#) **Submitter:** Hewill Kang **Opened:** 2022-06-01 **Last modified:** 2022-07-15

Priority: Not Prioritized

Discussion:

Currently, the size function of `chunk_view::outer-iterator::value_type` returns the result of `ranges::min`, since the operands are of type `range_difference_t<V>`, this will return a signed type, which is inconsistent with the return type of size of the forward-version of `chunk_view::iterator::value_type` (26.7.24.7 [\[range.chunk.fwd.iter\]](#)), which always returns an unsigned type.

I think it's more reasonable to return an unsigned type, since this is intentional behavior and doesn't fall back to using the default `view_interface::size`. And if LWG [3646](#) is eventually adopted, there's no reason why it shouldn't be made unsigned.

[2022-06-21; Reflector poll]

Set status to Tentatively Ready after five votes in favour during reflector poll.

[2022-07-15; LWG telecon: move to Ready]

Proposed resolution:

This wording is relative to [N4910](#).

1. Modify 26.7.24.4 [\[range.chunk.outer.value\]](#) as indicated:

```
constexpr auto size() const
    requires sized_sentinel_for<sentinel_t<V>, iterator_t<V>>;
```

-4- Effects: Equivalent to:

```
return to-unsigned-like(ranges::min(parent_>remainder_, ranges::end(parent_>base_) - *parent_>current_));
```

3708. `take_while_view::sentinel`'s conversion constructor should move

Section: 26.7.9.3 [[range.take_while.sentinel](#)] **Status:** [Ready](#) **Submitter:** Hewill Kang **Opened:** 2022-06-03 **Last modified:** 2022-07-15

Priority: Not Prioritized

Discussion:

The conversion constructor of `take_while_view::sentinel` requires `sentinel_t<V>` must satisfy `convertible_to<sentinel_t<Base>>`, which indicates that the rvalue reference of `sentinel_t<V>` can be converted to `sentinel_t<Base>`, but in the *Effects* element, we assign the lvalue `s.end_` to `end_`.

[2022-06-21; Reflector poll]

Set status to Tentatively Ready after seven votes in favour during reflector poll.

[2022-07-15; LWG telecon: move to Ready]

Proposed resolution:

This wording is relative to [N4910](#).

1. Modify 26.7.9.3 [[range.take_while.sentinel](#)] as indicated:

```
constexpr sentinel(sentinel<!Const> s)
    requires Const && convertible_to<sentinel_t<V>, sentinel_t<Base>>;

-2- Effects: Initializes end_ with std::move(s.end_) and pred_ with s.pred_.
```

3709. LWG-3703 was underly ambitious

Section: 22.5.3.1 [[optional.optional.general](#)], 22.6.3.1 [[variant.variant.general](#)] **Status:** [Ready](#) **Submitter:** Casey Carter **Opened:** 2022-06-08 **Last modified:** 2022-07-15

Priority: Not Prioritized

Discussion:

LWG 3703 struck redundant language from 22.8.7.1 [[expected.void.general](#)] specifying that (1) space allocated for an object is suitably aligned, and (2) a member annotated *// exposition only* in a class synopsis "is provided for exposition only."

Let's also strike similar occurrences from the wording for `optional` and `variant`.

[2022-06-21; Reflector poll]

Set status to Tentatively Ready after nine votes in favour during reflector poll.

[2022-07-15; LWG telecon: move to Ready]

Proposed resolution:

This wording is relative to [N4910](#).

1. Modify 22.5.3.1 [[optional.optional.general](#)] as indicated:

-1- Any instance of `optional<T>` at any given time either contains a value or does not contain a value. When an instance of `optional<T>` *contains a value*, it means that an object of type `T`, referred to as the optional object's *contained value*, is allocated within the storage of the optional object. Implementations are not permitted to use additional storage, such as dynamic memory, to allocate its contained value. ~~The contained value shall be allocated in a region of the `optional<T>` storage suitably aligned for the type `T`.~~ When an object of type `optional<T>` is contextually converted to `bool`, the conversion returns `true` if the object contains a value; otherwise the conversion returns `false`.

-2- ~~Member `val` is provided for exposition only.~~ When an `optional<T>` object contains a value, `member` `val` points to the contained value.

2. Modify 22.6.3.1 [[variant.variant.general](#)] as indicated:

-1- Any instance of `variant` at any given time either holds a value of one of its alternative types or holds no value. When an instance of `variant` holds a value of alternative type `T`, it means that a value of type `T`, referred to as the variant object's *contained value*, is allocated within the storage of the variant object. Implementations are not permitted to use additional storage, such as dynamic memory, to allocate the contained value. ~~The contained value shall be allocated in a region of the variant storage suitably aligned for all types in `Types`.~~

3710. The end of `chunk_view` for input ranges can be `const`

Section: 26.7.24.2 [\[range.chunk.view.input\]](#) **Status:** [Ready](#) **Submitter:** Hewill Kang **Opened:** 2022-06-09 **Last modified:** 2022-07-15

Priority: Not Prioritized

View other [active issues](#) in [\[range.chunk.view.input\]](#).

View all other [issues](#) in [\[range.chunk.view.input\]](#).

Discussion:

The input range version of `chunk_view`'s `end` is a very simple function that only returns `default_sentinel`, and simple ends like this also appear in other range adaptors, such as `basic_istream_view`, `lazy_split_view::outer_iterator::value_type`, and `chunk_view::outer_iterator::value_type`.

However, unlike `chunk_view`, their ends all are `const`-qualified, which allows us to freely get `default_sentinel` through the end of these `const` objects even though they may not themselves be ranges.

I think we should add `const` to this `chunk_view`'s `end` as I don't see any harm in doing this, and in some cases, it may have a certain value. Also, this makes it consistent with `basic_istream_view` and the upcoming `std::generator`, which, like it, only has a non-`const` `begin`.

[2022-06-21; Reflector poll]

Set status to Tentatively Ready after eight votes in favour during reflector poll.

[2022-07-15; LWG telecon: move to Ready]

Proposed resolution:

This wording is relative to [N4910](#).

1. Modify 26.7.24.2 [\[range.chunk.view.input\]](#) as indicated:

```
namespace std::ranges {
    [...]

    template<view V>
        requires input_range<V>
    class chunk_view : public view_interface<chunk_view<V>> {
        V base_ = V();
        [...] // exposition only
    public:
        [...]

        constexpr outer_iterator begin();
        constexpr default_sentinel_t end() const noexcept;

        constexpr auto size() requires sized_range<V>;
        constexpr auto size() const requires sized_range<const V>;
    };
    [...]
}

[...]

constexpr default_sentinel_t end() const noexcept;

-4- Returns: default_sentinel.
```

[3711](#). Missing preconditions for `slide_view` constructor

Section: 26.7.25.2 [\[range.slide.view\]](#) **Status:** [Ready](#) **Submitter:** Hewill Kang **Opened:** 2022-06-10 **Last modified:** 2022-07-15

Priority: Not Prioritized

Discussion:

It doesn't make sense for `slide_view` when `n` is not positive, we should therefore add a precondition for its constructor just like we did for `chunk_view`.

Daniel:

Indeed the accepted paper describes the intention in [P2442R1 Section 4.2](#) by saying that "It is a precondition that `N` is positive" in the design wording but omitted to add a normative precondition in the proposed wording.

[2022-06-21; Reflector poll]

Set status to Tentatively Ready after six votes in favour during reflector poll.

[2022-07-15; LWG telecon: move to Ready]

Proposed resolution:

This wording is relative to [N4910](#).

1. Modify 26.7.25.2 [\[range.slide.view\]](#) as indicated:

```
constexpr explicit slide_view(V base, range_difference_t<V> n);
```

-?- Preconditions: `n > 0` is true.

-1- Effects: Initializes `base_` with `std::move(base)` and `n_` with `n`.

3712. chunk_view and slide_view should not be default_initializable

Section: 26.7.24.2 [\[range.chunk.view.input\]](#), 26.7.24.6 [\[range.chunk.view.fwd\]](#), 26.7.25.2 [\[range.slide.view\]](#) **Status:** [Ready](#) **Submitter:** Hewill Kang
Opened: 2022-06-10 **Last modified:** 2022-07-15

Priority: Not Prioritized

View other [active issues](#) in [\[range.chunk.view.input\]](#).

View all other [issues](#) in [\[range.chunk.view.input\]](#).

Discussion:

Both `chunk_view` and `slide_view` have a precondition that `N` must be positive, but they are still `default_initializable` when the underlying range is `default_initializable`, which makes the member variable `n_` initialized with an invalid value `0` when they are default-constructed, which produces the following unexpected result:

```
#include <ranges>

using V = std::ranges::iota_view<int, int>;
static_assert(std::ranges::slide_view<V>().empty()); // fails
static_assert(std::ranges::chunk_view<V>().empty()); // division by zero is not a constant expression
```

Although we could provide a default positive value for `n_`, I think a more appropriate solution would be to not provide the default constructor, since default-constructed values for integer types will never be valid.

[2022-06-21; Reflector poll]

Set status to Tentatively Ready after six votes in favour during reflector poll.

[2022-07-15; LWG telecon: move to Ready]

Proposed resolution:

This wording is relative to [N4910](#).

1. Modify 26.7.24.2 [\[range.chunk.view.input\]](#) as indicated:

```
namespace std::ranges {
    [...]

    template<view V>
    requires input_range<V>
    class chunk_view : public view_interface<chunk_view<V>> {
        V base_ = V{}; // exposition only
        range_difference_t<V> n_ = 0; // exposition only
        range_difference_t<V> remainder_ = 0; // exposition only
        [...]
    public:
        chunk_view() requires default_initializable<V> = default;
        constexpr explicit chunk_view(V base, range_difference_t<V> n);
        [...]
    };
    [...]
}
```

2. Modify 26.7.24.6 [\[range.chunk.view.fwd\]](#) as indicated:

```
namespace std::ranges {
    template<view V>
    requires forward_range<V>
    class chunk_view<V> : public view_interface<chunk_view<V>> {
        V base_ = V{}; // exposition only
        range_difference_t<V> n_ = 0; // exposition only
        [...]
    public:
        chunk_view() requires default_initializable<V> = default;
        constexpr explicit chunk_view(V base, range_difference_t<V> n);
        [...]
    };
    [...]
}
```

```
};
}
```

3. Modify 26.7.25.2 [\[range.slide.view\]](#) as indicated:

```
namespace std::ranges {
    [...]

    template<forward_range V>
    requires view<V>
    class slide_view : public view_interface<slide_view<V>> {
        V base_ = V{}; // exposition only
        range_difference_t<V> n_ = 0; // exposition only
        [...]
    public:
        slide_view() requires default_initializable<V> = default;
        constexpr explicit slide_view(V base, range_difference_t<V> n);

        [...]
    };
    [...]
}
```

3713. Sorted with respect to comparator (only)

Section: 27.8.1 [\[alg.sorting.general\]](#) **Status:** [Ready](#) **Submitter:** Casey Carter **Opened:** 2022-06-10 **Last modified:** 2022-07-15

Priority: Not Prioritized

Discussion:

[P0896R4](#) changed the term of art "sorted with respect to comparator" defined in 27.8.1 [\[alg.sorting.general\]](#) paragraph 5 to "sorted with respect to comparator and projection." That proposal updated the algorithm specifications consistently. However, there were uses of the old term outside of 27 [\[algorithms\]](#) that are now without meaning. We should bring back the term "sorted with respect to comparator" to fix that lack.

[2022-06-21; Reflector poll]

Set status to Tentatively Ready after eight votes in favour during reflector poll.

[2022-07-15; LWG telecon: move to Ready]

Proposed resolution:

This wording is relative to [N4910](#).

1. Modify 27.8.1 [\[alg.sorting.general\]](#) as indicated:

-5- A sequence is *sorted with respect to a comp and proj* for a comparator and projection comp and proj if for every iterator i pointing to the sequence and every non-negative integer n such that i + n is a valid iterator pointing to an element of the sequence,

```
bool(invoker(comp, invoker(proj, *(i + n)), invoker(proj, *i)))
```

is false.

-?- A sequence is *sorted with respect to a comparator comp* for a comparator comp if it is sorted with respect to comp and identity{. (the identity projection).

3715. view_interface::empty is overconstrained

Section: 26.5.3.1 [\[view.interface.general\]](#) **Status:** [Ready](#) **Submitter:** Hewill Kang **Opened:** 2022-06-12 **Last modified:** 2022-07-15

Priority: Not Prioritized

View other [active issues](#) in [view.interface.general].

View all other [issues](#) in [view.interface.general].

Discussion:

Currently, view_interface::empty has the following constraints

```
constexpr bool empty() requires forward_range<D> {
    return ranges::begin(derived()) == ranges::end(derived());
}
```

which seems reasonable, since we need to guarantee the equality preservation of the expression ranges::begin(r).

However, this prevents a more efficient way in some cases, i.e., when `D` models `sized_range`, we only need to determine whether the value of `ranges::size` is 0. Since `sized_range` and `forward_range` are orthogonal to each other, this also prevents any range that models `sized_range` but not `forward_range`.

Consider:

```
#include <iostream>
#include <ranges>

int main() {
    auto f = std::views::iota(0, 5)
        | std::views::filter([](int) { return true; });
    auto r = std::views::counted(f.begin(), 4)
        | std::views::slide(2);
    std::cout << (r.size() == 0) << "\n"; // #1
    std::cout << r.empty() << "\n";      // #2, calls r.begin() == r.end()
}
```

Since `r` models `sized_range`, #1 will invoke `slide_view::size`, which mainly invokes `ranges::distance`; However, #2 invokes `view_interface::empty` and evaluates `r.begin() == r.end()`, which constructs the iterator, invokes `ranges::next`, and caches the result, which is unnecessary.

Also consider:

```
#include <iostream>
#include <ranges>

int main() {
    auto i = std::views::istream<int>(std::cin);
    auto r = std::views::counted(i.begin(), 4)
        | std::views::chunk(2);
    std::cout << (r.size() == 0) << "\n"; // #1
    std::cout << !r << "\n";           // #2, equivalent to r.size() == 0
    std::cout << r.empty() << "\n";    // #3, ill-formed
}
```

Since `r` is still `sized_range`, #1 will invoke `chunk_view::size`. #2 is also well-formed since `view_interface::operator bool` only requires the expression `ranges::empty(r)` to be well-formed, which first determines the validity of `r.empty()`, and ends up evaluating #1; However, #3 is ill-formed since `r` is not a `forward_range`.

Although we can still use `ranges::empty` to determine whether `r` is empty, this inconsistency of the validity of `!r` and `r.empty()` is quite unsatisfactory.

I see no reason to prevent `view_interface::empty` when `D` is `sized_range`, since checking whether `ranges::size(r) == 0` is an intuitive way to check for empty, as `ranges::empty` does.

[2022-06-21; Reflector poll]

Set status to Tentatively Ready after six votes in favour during reflector poll.

[2022-07-15; LWG telecon: move to Ready]

Proposed resolution:

This wording is relative to [N4910](#).

1. Modify 26.5.3.1 [\[view.interface.general\]](#) as indicated:

```
namespace std::ranges {
    template<class D>
        requires is_class_v<D> && same_as<D, remove_cv_t<D>>
        class view_interface {
        private:
            constexpr D& derived() noexcept {                // exposition only
                return static_cast<D&>(*this);
            }
            constexpr const D& derived() const noexcept {    // exposition only
                return static_cast<const D&>(*this);
            }
        public:
            constexpr bool empty() requires sized_range<D> || forward_range<D> {
                if constexpr (sized_range<D>)
                    return ranges::size(derived()) == 0;
                else
                    return ranges::begin(derived()) == ranges::end(derived());
            }
            constexpr bool empty() const requires sized_range<const D> || forward_range<const D> {
                if constexpr (sized_range<const D>)
                    return ranges::size(derived()) == 0;
                else
                    return ranges::begin(derived()) == ranges::end(derived());
            }
        };
    [...]
```

```
};  
}
```

3719. Directory iterators should be usable with default sentinel

Section: 31.12.11.1 [\[fs.class.directory_iterator.general\]](#), 31.12.12.1 [\[fs.class.rec.dir.itr.general\]](#) **Status:** [Ready](#) **Submitter:** Jonathan Wakely **Opened:** 2022-06-17 **Last modified:** 2022-07-15

Priority: Not Prioritized

Discussion:

We added comparisons with `default_sentinel_t` to the stream and streambuf iterators, because their past-the-end iterator is just a default-constructed iterator. We didn't do the same for filesystem directory iterators, but they also use a default-constructed value as the sentinel.

The proposed resolution addresses this oversight.

Previous resolution [SUPERSEDED]:

This wording is relative to [N4910](#).

1. Modify 31.12.11.1 [\[fs.class.directory_iterator.general\]](#), class `directory_iterator` synopsis, as indicated:

```
namespace std::filesystem {  
    class directory_iterator {  
        [...]  
  
        const directory_entry& operator*() const;  
        const directory_entry* operator->() const;  
        directory_iterator& operator++();  
        directory_iterator& increment(error_code& ec);  
  
        friend bool operator==(const directory_iterator& lhs, default_sentinel_t) noexcept  
        { return lhs == end(lhs); }  
  
        // other members as required by 25.3.5.3 \[input.iterators\], input iterators  
    };  
}
```

2. Modify 31.12.12.1 [\[fs.class.rec.dir.itr.general\]](#), class `recursive_directory_iterator` synopsis, as indicated:

```
namespace std::filesystem {  
    class recursive_directory_iterator {  
        [...]  
  
        void pop();  
        void pop(error_code& ec);  
        void disable_recursion_pending();  
  
        friend bool operator==(const recursive_directory_iterator& lhs, default_sentinel_t) noexcept  
        { return lhs == end(lhs); }  
  
        // other members as required by 25.3.5.3 \[input.iterators\], input iterators  
    };  
}
```

[2022-07-06; Jonathan Wakely revises proposed resolution and adds regex iterators as suggested on the reflector.]

[2022-07-11; Reflector poll]

Set status to Tentatively Ready after six votes in favour during reflector poll.

[2022-07-15; LWG telecon: move to Ready]

Proposed resolution:

This wording is relative to [N4910](#).

1. Modify 31.12.11.1 [\[fs.class.directory_iterator.general\]](#), class `directory_iterator` synopsis, as indicated:

```
namespace std::filesystem {  
    class directory_iterator {  
        [...]  
  
        const directory_entry& operator*() const;  
        const directory_entry* operator->() const;  
        directory_iterator& operator++();  
        directory_iterator& increment(error_code& ec);  
    };  
}
```

```

    bool operator==(default_sentinel_t) const noexcept
    { return *this == directory_iterator(); }

    // other members as required by 25.3.5.3 [input.iterators], input iterators
};
}

```

2. Modify 31.12.12.1 [fs.class.rec.dir.itr.general], class recursive_directory_iterator synopsis, as indicated:

```

namespace std::filesystem {
    class recursive_directory_iterator {
    [...]

    void pop();
    void pop(error_code& ec);
    void disable_recursion_pending();

    bool operator==(default_sentinel_t) const noexcept
    { return *this == recursive_directory_iterator(); }

    // other members as required by 25.3.5.3 [input.iterators], input iterators
};
}

```

3. Modify 32.11.1.1 [re.regiter.general], regex_iterator synopsis, as indicated:

```

namespace std {
    template<class BidirectionalIterator,
            class charT = typename iterator_traits<BidirectionalIterator>::value_type,
            class traits = regex_traits<charT>>
    class regex_iterator {
    [...]
    regex_iterator& operator=(const regex_iterator&);
    bool operator==(const regex_iterator&) const;
    bool operator==(default_sentinel_t) const { return *this == regex_iterator(); }
    const value_type& operator*() const;
    const value_type* operator->() const;
    };
}

```

4. Modify 32.11.2.1 [re.tokiter.general], regex_token_iterator synopsis, as indicated:

```

namespace std {
    template<class BidirectionalIterator,
            class charT = typename iterator_traits<BidirectionalIterator>::value_type,
            class traits = regex_traits<charT>>
    class regex_token_iterator {
    [...]
    regex_iterator& operator=(const regex_token_iterator&);
    bool operator==(const regex_token_iterator&) const;
    bool operator==(default_sentinel_t) const { return *this == regex_token_iterator(); }
    const value_type& operator*() const;
    const value_type* operator->() const;
    };
}

```

3721. Allow an *arg-id* with a value of zero for *width* in *std-format-spec*

Section: 22.14.2.2 [format.string.std] Status: [Ready](#) Submitter: Mark de Wever Opened: 2022-06-19 Last modified: 2022-07-15

Priority: 3

View other [active issues](#) in [format.string.std].

View all other [issues](#) in [format.string.std].

Discussion:

Per 22.14.2.2 [format.string.std]/7

If { *arg-id*_{opt} } is used in a *width* or *precision*, the value of the corresponding formatting argument is used in its place. If the corresponding formatting argument is not of integral type, or its value is negative for *precision* or non-positive for *width*, an exception of type `format_error` is thrown.

During a libcpp code review Victor mentioned it would be nice to allow zero as a valid value for the *arg-id* when used for the *width*. This would simplify the code by having the same requirements for the *arg-id* for the *width* and *precision* fields. A *width* of zero has no effect on the output.

In the *std-format-spec* the *width* is restricted to a positive-integer to avoid parsing ambiguity with the zero-padding option. This ambiguity doesn't happen using an *arg-id* with the value zero. Therefore I only propose to change the *arg-id*'s requirement.

Note the Standard doesn't specify the *width* field's effect on the output. Specifically [tab:format.align] doesn't refer to the *width* field. This is one of the items addressed by [P2572](#). The proposed resolution works in combination with the wording of that paper.

[2022-07-08; Reflector poll]

Set priority to 3 after reflector poll.

[2022-07-11; Reflector poll]

Set status to Tentatively Ready after seven votes in favour during reflector poll.

[2022-07-15; LWG telecon: move to Ready]

Proposed resolution:

This wording is relative to [N4910](#).

1. Modify 22.14.2.2 [\[format.string.std\]](#) as indicated:

-7- If { *arg-id_{opt}* } is used in a *width* or *precision*, the value of the corresponding formatting argument is used in its place. If the corresponding formatting argument is not of integral type, or its value is negative ~~for precision or non-positive for width~~, an exception of type `format_error` is thrown.

[3724](#). *decay-copy* should be constrained

Section: 16.3.3.2 [\[expos.only.func\]](#) **Status:** [Ready](#) **Submitter:** Hui Xie **Opened:** 2022-06-23 **Last modified:** 2022-07-15

Priority: 3

View all other [issues](#) in [\[expos.only.func\]](#).

Discussion:

The spec of `views::all` 26.7.5.1 [\[range.all.general\]](#) p2 says:

Given a subexpression *E*, the expression `views::all(E)` is expression-equivalent to:

(2.1) — *decay-copy*(*E*) if the decayed type of *E* models `view`.

[...]

If *E* is an lvalue move-only view, according to the spec, `views::all(E)` would be expression-equivalent to *decay-copy*(*E*).

However, 16.3.3.2 [\[expos.only.func\]](#) p2 defines *decay-copy* as follows

```
template<class T> constexpr decay_t<T> decay-copy(T&& v)
noexcept(is_nothrow_convertible_v<T, decay_t<T>>) // exposition only
{ return std::forward<T>(v); }
```

It is unconstrained. As a result, for the above example, `views::all(E)` is a well-formed expression and it would only error on the template instantiation of the function body of *decay-copy*, because *E* is an lvalue of move-only type and not copyable.

I think this behaviour is wrong, instead, we should make *decay-copy*(*E*) ill-formed if it is not copyable, so that `views::all(E)` can be SFINAE friendly.

[2022-07-08; Reflector poll]

Set priority to 3 after reflector poll.

[2022-07-11; Reflector poll]

Set status to Tentatively Ready after seven votes in favour during reflector poll.

[2022-07-15; LWG telecon: move to Ready]

Proposed resolution:

This wording is relative to [N4910](#).

1. Modify 16.3.3.2 [\[expos.only.func\]](#) as indicated:

-2- The following are defined for exposition only to aid in the specification of the library:

```
namespace std {
template<class T>
requires convertible_to<T, decay_t<T>>
constexpr decay_t<T> decay-copy(T&& v)
noexcept(is_nothrow_convertible_v<T, decay_t<T>>) // exposition only
{ return std::forward<T>(v); }
[...]
```